

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II



SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE
DELL'INFORMAZIONE

CORSO DI LAUREA TRIENNALE IN INFORMATICA

SVILUPPO DI UN' APP IOS PER L'APPRENDIMENTO COLLABORATIVO

Relatore

Prof. Francesco ISGRÒ

Candidato

Giovanni Luca DI MAIO

N86/2036

Anno Accademico 2025–2026

Indice

Introduzione	6
Capitolo 1	8
Idea e metodo di apprendimento	8
1.1 Apple Developer Academy	8
1.2 Challenge Based Learning (CBL).....	8
1.3 Il Problema	9
1.4 L'applicazione IOS	9
1.5 Evoluzione e attualità del progetto nel panorama post-pandemico	10
Capitolo 2	12
Tecnologie ed API	12
2.1 Swift	12
2.2 XCode	13
2.3 CloudKit.....	13
2.4 UIKit	14
2.5 VisionKit e Vision	14
2.6 MessageUI	14
Capitolo 3	15
Documento di analisi e specifica dei requisiti.....	15
3.1 Requisiti funzionali	15
3.2 Requisiti non funzionali.....	16
3.3 Requisiti di dominio	16
3.4 Use Case Diagram	18

3.5 Mock-Up	19
3.6 Tabelle di Cockburn	24
3.6.1 Tabella di Cockburn per l’inserimento di un nuovo testo.....	24
3.6.2 Tabella di Cockburn per l’aggiunta di un nuovo commento.....	27
3.6.3 Tabella di Cockburn per l’eliminazione di un testo	29
3.6.4 Tabella di Cockburn per la condivisione di un testo	31
3.7 Class Diagram del database	32
Capitolo 4.....	34
Software design	34
4.1 Design Pattern	34
4.2 System Design : L'architettura MVC in Swift	35
4.3 Object Design.....	37
4.3.1 Architettura Statica: Class Diagram Globale	37
4.3.2 Flusso di Inserimento e Scansione (Activity e State Chart)	39
4.3.3 Dinamica dell'Acquisizione OCR (Sequence Diagram).....	42
4.3.4 Inserimento di un Nuovo Testo su CloudKit	43
4.3.5 Inserimento di un Commento Semantico	44
4.3.6 Sincronizzazione e Visualizzazione dei Testi.....	45
4.3.7 Eliminazione a Cascata di un Testo	46
Capitolo 5.....	47
Testing e piano di testing	47
5.1 Unit Testing	47
5.1.1 Testing del metodo di validazione input (SendMailViewController)	48
5.2 TestFlight Beta Testing e Validazione UX.....	50
5.2.1 Fase 1: Prima Iterazione e Bug Fixing.....	51
5.2.2 Fase 2: Distribuzione su Larga Scala e Telemetria	51

5.2.3 Valutazione dei Risultati e Sondaggio UX	52
Conclusioni.....	56
Bibliografia.....	58
Ringraziamenti	60

Elenco delle figure

Figura 1:Use Case Diagram.....	18
Figura 2:Mock-Up Homepage	19
Figura 3:Mock-Up Visualizzazione testo e commenti.....	20
Figura 4:Mock-Up Inserimento testo.....	21
Figura 5:Mock-Up Inserimento commento	22
Figura 6:Eliminazione testo/commento	23
Figura 7: Class diagram del database	32
Figura 8:Modello MVC	35
Figura 9: Modello MVC in Swift	36
Figura 10: Class Diagram Globale dell'applicazione.....	38
Figura 11: Activity Diagram per la funzionalità di inserimento testo	40
Figura 12: State Chart Diagram per il processo di scansione e salvataggio	41
Figura 13: Sequence Diagram del flusso OCR tramite VisionKit	42
Figura 14: Sequence Diagram per il salvataggio di un nuovo testo	43
Figura 15: Sequence Diagram per l'inserimento di un commento.....	44
Figura 16: Sequence Diagram per il recupero dei dati da CloudKit.....	45
Figura 17: Sequence Diagram per la funzionalità di Eliminazione a Cascata.....	46
Figura 18: Codice del metodo textView	48
Figura 19: Codice dei test	49
Figura 20: GFC del metodo textView.....	50
Figura 21: Istogramma della distribuzione dei voti sull'intuitività dell'interfaccia.	54
Figura 22: Istogramma della distribuzione dei voti sull' interazione e commento.....	54
Figura 23: Istogramma della distribuzione dei voti sull'affidabilità dello Scanner OCR	55

Introduzione

Il presente lavoro di tesi documenta la progettazione e lo sviluppo di un'applicazione mobile per ambiente iOS, nata all'interno della cornice collaborativa della Apple Developer Academy. Il progetto mira a supportare e potenziare le attività di studio cooperativo, offrendo agli studenti uno strumento digitale per la condivisione e la risoluzione di dubbi accademici da remoto.

L'idea progettuale trae ispirazione dalle sfide poste dall'emergenza pandemica, che ha evidenziato la necessità di piattaforme di E-Learning capaci di andare oltre la semplice trasmissione di contenuti, promuovendo una partecipazione attiva. L'applicazione consente di gestire documenti testuali, arricchendoli con analisi e commenti puntuali su specifiche porzioni di testo.

Un elemento distintivo del software è l'integrazione di tecnologie avanzate di Computer Vision per l'acquisizione dei testi. Grazie all'utilizzo di scanner OCR nativi, l'utente può digitalizzare istantaneamente testi cartacei, rendendoli pronti per l'analisi e la condivisione.

Lo sviluppo dell'applicativo è stato guidato dai principi cardine dell'Ingegneria del Software [6], con l'obiettivo di garantire un prodotto non solo funzionale, ma anche affidabile, manutenibile e scalabile.

L'elaborato è strutturato come segue:

- **Capitolo 1:** Descrizione del contesto operativo (Apple Developer Academy), della metodologia di apprendimento (Challenge Based Learning) e genesi dell'idea.

- **Capitolo 2:** Analisi delle tecnologie, dei linguaggi (Swift) e delle API (CloudKit, VisionKit) utilizzate.
- **Capitolo 3:** Documentazione di analisi e specifica dei requisiti, corredata da Use Case e prototipazione (Mock-up).
- **Capitolo 4:** Architettura del sistema e Software Design, con focus sui design pattern adottati.
- **Capitolo 5:** Piano di testing, validazione del software e risultati ottenuti.

Capitolo 1

Idea e metodo di apprendimento

Il presente capitolo approfondisce il contesto operativo e metodologico in cui è maturato il progetto. Verrà analizzata l'esperienza formativa presso l'Apple Developer Academy e l'adozione del framework didattico Challenge Based Learning (CBL). Saranno inoltre esaminati i presupposti analitici che, partendo dall'analisi delle criticità emerse durante l'emergenza pandemica, hanno condotto alla definizione della sfida progettuale e alla successiva ideazione dell'applicativo.

1.1 Apple Developer Academy

Il presente lavoro di tesi è frutto dell'esperienza maturata durante l'anno accademico 2019/2020 presso la Apple Developer Academy, un centro di eccellenza nato dalla collaborazione tra l'Università degli Studi di Napoli Federico II e Apple. Il percorso, della durata di nove mesi, è focalizzato sull'intero ciclo di vita del software: dalla concezione del prodotto alla sua pubblicazione su App Store, passando per una rigorosa fase di analisi e progettazione. L'ambiente, fortemente improntato alla collaborazione in team multidisciplinari, ha permesso di affrontare sfide di complessità variabile, culminando nello sviluppo del progetto finale qui documentato.

1.2 Challenge Based Learning (CBL)

L'approccio pedagogico adottato è il Challenge Based Learning (CBL). Diversamente dai metodi tradizionali, il CBL spinge gli studenti a risolvere problemi reali partendo da una 'sfida' (Challenge) aperta [5]. Il processo si articola in tre macro-fasi:

- 1 **Engage:** Trasformazione di un'idea astratta in una sfida concreta attraverso 'guiding questions'.
- 2 **Investigate:** Ricerca attiva di soluzioni tramite interviste, analisi di mercato e studio della documentazione.
- 3 **Act:** Implementazione della soluzione e valutazione dei risultati basata sul feedback degli utenti.

1.3 Il Problema

Durante la fase di *Engage*, l'attenzione si è focalizzata sull'E-Learning, settore diventato critico a causa della chiusura delle scuole durante l'emergenza pandemica. Dalle ricerche è emerso che, oltre alle carenze infrastrutturali (con oltre 1200 comuni italiani privi di una copertura di rete stabile), uno dei problemi principali risiedeva nella mancanza di strumenti che incentivassero l'Apprendimento Cooperativo da remoto. La sfida è stata quindi quella di creare un ponte digitale tra gli studenti per facilitare lo studio domestico.

1.4 L'applicazione IOS

L'obiettivo primario del software è incentivare lo studio domestico e la collaborazione tra pari. L'app si presenta come un hub documentale dove lo studente può organizzare i propri testi di studio, categorizzandoli tramite Titoli e #TAG.

L'inserimento di un nuovo documento è stato progettato per essere estremamente rapido. L'utente può scegliere tra l'inserimento manuale o la **scansione intelligente**. Come mostrato nella prototipazione e nelle demo tecniche, l'attivazione della fotocamera permette di estrarre il testo da libri o appunti fisici, popolando automaticamente l'area di lavoro ed eliminando la necessità di trascrizioni manuali onerose.

Una volta creato il testo, l'architettura abilita la funzione core: il **commento semantico**. Attraverso una selezione precisa della porzione di testo (evidenziazione), l'utente può allegare una nota, una traduzione o una spiegazione. Tutti i dati sono sincronizzati in tempo reale tramite **CloudKit**, garantendo che ogni aggiornamento sia persistente e disponibile su tutti i dispositivi dell'utente.

1.5 Evoluzione e attualità del progetto nel panorama post-pandemico

Sebbene l'idea originaria dell'applicazione sia maturata durante l'emergenza sanitaria del 2020 per sopperire alle limitazioni della Didattica a Distanza (DAD), l'architettura e le funzionalità del software risultano oggi perfettamente allineate ai principali macro-trend del settore EdTech per il biennio 2025-2026 [8].

La fine dell'emergenza non ha segnato un ritorno esclusivo alla didattica analogica, bensì la transizione verso modelli consolidati di Blended Learning (apprendimento ibrido), diventati la norma strategica che unisce l'istruzione faccia a faccia agli strumenti digitali [9]. A livello universitario, i report indicano un incremento permanente dei programmi di studio in modalità mista [11]. In questo contesto odierno, Stratum si colloca come una soluzione altamente attuale grazie a due fattori chiave:

1. Il ponte tra fisico e digitale tramite Computer Vision (OCR)

Con il ritorno nelle aule, gli studenti utilizzano massicciamente supporti fisici (libri di testo, dispense stampate, appunti cartacei). La funzione di scansione intelligente integrata nell'app tramite VisionKit risolve il problema della frammentazione dei supporti: permette di digitalizzare istantaneamente il materiale fisico, rendendolo immediatamente indicizzabile, ricercabile e pronto per essere

condiviso. Questo approccio ibrido è fondamentale nelle attuali dinamiche di studio, dove la velocità di acquisizione del dato cartaceo per la successiva rielaborazione digitale costituisce un forte vantaggio.

2. Social Learning e superamento dell'isolamento digitale

Le recenti analisi di mercato sulle tecnologie educative indicano il Social Learning e i modelli ibridi come gli elementi centrali dell'era post-pandemica [7]. L'apprendimento sociale costituisce infatti la base delle pedagogie emergenti orientate alla collaborazione [10]. La gestione nativa dei commenti semantici e la condivisione gerarchica garantita da CloudKit permettono a Stratum di rispondere esattamente a questa esigenza. L'applicativo non si limita a essere un archivio passivo di documenti, ma si trasforma in un vero e proprio spazio di interazione, in cui un dubbio sollevato su una specifica porzione di testo può essere risolto in modo cooperativo da altri colleghi, replicando e potenziando le dinamiche dei gruppi di studio tradizionali.

In conclusione, l'applicativo dimostra una forte resilienza progettuale: concepito inizialmente per abbattere le barriere fisiche del lockdown, si configura oggi come uno strumento ottimizzato per ecosistemi di apprendimento ibridi, promuovendo la partecipazione attiva e l'inclusività tecnologica.

Capitolo 2

Tecnologie ed API

In questo capitolo vengono esaminate le tecnologie e le interfacce di programmazione (**API**) impiegate per la realizzazione dell'applicativo. Partendo dall'analisi del linguaggio di programmazione **Swift** e dell'ambiente di sviluppo **Xcode**, la trattazione si estenderà ai framework di sistema che hanno permesso di implementare le funzionalità core del software, con particolare riferimento alla persistenza dei dati, alla sincronizzazione in cloud e all'integrazione di moduli di **Computer Vision**.

2.1 Swift

Il linguaggio di programmazione adottato è **Swift**, un linguaggio multi-paradigma sviluppato da Apple per i propri sistemi operativi. La scelta di Swift è stata dettata da diverse caratteristiche tecniche che lo rendono superiore ai predecessori (come Objective-C) nel contesto dello sviluppo moderno:

- **Sicurezza (Type Safety):** Swift è un linguaggio fortemente tipizzato che riduce drasticamente gli errori a tempo di esecuzione, grazie a una gestione rigorosa dei tipi e degli opzionali.
- **Performance:** Grazie al compilatore LLVM, Swift offre prestazioni elevate, risultando sensibilmente più veloce rispetto a linguaggi interpretati come Python o allo stesso Objective-C in determinati task computazionali.
- **Gestione della memoria:** L'utilizzo dell'**Automatic Reference Counting (ARC)** garantisce una gestione efficiente della memoria, liberando automaticamente le risorse non più necessarie e ottimizzando le prestazioni complessive del sistema.

- **Sintassi Moderna:** La vicinanza al linguaggio naturale facilita la manutenibilità del codice, permettendo al contempo l'implementazione nativa di design pattern complessi come Observer e Strategy, centrali nell'architettura MVC dell'app [1].

2.2 XCode

Xcode rappresenta l'ambiente di sviluppo integrato (IDE) primario per l'ecosistema Apple. Oltre a fornire i compilatori necessari, Xcode integra strumenti di diagnostica e debugging avanzati. Durante lo sviluppo dell'app, è stato fondamentale l'utilizzo di strumenti come **l'Interface Builder** per la prototipazione delle view e il simulatore integrato per la verifica del comportamento del software su diversi modelli di iPhone.

2.3 CloudKit

Per la persistenza e la sincronizzazione dei dati, il progetto si affida interamente a **CloudKit** come backend as a service (BaaS). Tutte le entità dell'applicazione (Testo e Commento) sono mappate su oggetti CKRecord e archiviate nel privateCloudDatabase dell'utente. L'impiego diretto delle API di CloudKit offre vantaggi significativi:

- **Autenticazione trasparente:** L'utente non necessita di registrazioni aggiuntive, poiché il sistema sfrutta nativamente le credenziali dell'Apple ID già configurato sul dispositivo.
- **Sicurezza e Privacy:** I dati sono sincronizzati sui server Apple nel pieno rispetto delle normative internazionali (incluso il GDPR), garantendo un elevato standard di affidabilità e protezione.

Oltre alla persistenza privata, CloudKit è stato sfruttato per implementare la condivisione gerarchica dei dati tramite oggetti CKShare [2].

2.4 UIKit

Il framework **UIKit** è stato impiegato per la costruzione dell'interfaccia utente. Esso fornisce l'architettura necessaria per la gestione degli eventi (tap, swipe action), la gerarchia delle viste e la modellazione delle schermate touch-screen. Elementi come le **UITableView** e i **UIAlertController** sono stati personalizzati per garantire un'esperienza utente fluida.

2.5 VisionKit e Vision

L'integrazione della funzionalità di scanner intelligente è resa possibile dalla combinazione di due framework:

- **VisionKit**: Tramite la classe **VNDocumentCameraViewController**, l'app accede all'interfaccia di sistema per la scansione documentale, gestendo automaticamente il rilevamento dei bordi e la correzione prospettica.
- **Vision**: L'immagine acquisita viene elaborata tramite una **VNRecognizeTextRequest**, che applica algoritmi di **OCR** (Optical Character Recognition) per estrarre stringhe di testo modificabili [3].

2.6 MessageUI

Il framework **MessageUI** è stato utilizzato per implementare un canale di supporto diretto per la segnalazione di bug. Attraverso l'uso del **MFMailComposeViewController**, l'applicazione apre un'interfaccia nativa precompilata, garantendo un metodo standardizzato e sicuro per la comunicazione tra utente e sviluppatore.

Capitolo 3

Documento di analisi e specifica dei requisiti

In questo capitolo viene definita la specifica dei requisiti del sistema, documento fondamentale che stabilisce formalmente le funzionalità e i vincoli del software. La trattazione include l'elencazione dei requisiti funzionali e non funzionali, la modellazione dei casi d'uso, la progettazione dell'interfaccia tramite mock-up e la definizione del modello logico dei dati attraverso il Class Diagram del database.

3.1 Requisiti funzionali

I requisiti funzionali descrivono i servizi che il sistema deve offrire in risposta a specifici input dell'utente. Per l'applicativo in esame, sono stati identificati i seguenti:

- **Inserimento Testo:** Il sistema deve permettere l'aggiunta di documenti testuali sia tramite input manuale (tastiera/incolla) sia tramite acquisizione intelligente con fotocamera.
- **Commento Semantico:** L'utente deve poter selezionare porzioni specifiche di un testo per associarvi commenti, note o spiegazioni.
- **Visualizzazione e Ricerca:** Il sistema deve garantire la visualizzazione dell'elenco dei testi salvati e permettere la ricerca filtrata per titolo o tag.
- **Cancellazione a Cascata:** Deve essere possibile eliminare un testo; tale operazione comporta la rimozione automatica di tutti i commenti associati.
- **Supporto Tecnico:** L'utente deve poter inviare segnalazioni di bug o feedback direttamente all'amministratore tramite email precompilata.

- **Condivisione e Collaborazione:** L'applicazione deve consentire la condivisione dei testi e dei relativi commenti con altri utenti tramite l'integrazione nativa delle API di CloudKit. Il sistema deve supportare la generazione di inviti sicuri, permettendo al proprietario di gestire i permessi di accesso (sola lettura o lettura/scrittura) e garantendo la sincronizzazione dei dati in tempo reale tra tutti i partecipanti alla sessione di studio.

3.2 Requisiti non funzionali

I requisiti non funzionali definiscono i vincoli tecnici e qualitativi del sistema:

- **Infrastruttura Cloud:** Il sistema deve utilizzare **CloudKit** come backend primario per la persistenza e la sincronizzazione multi-dispositivo.
- **Interfaccia Utente:** L'applicazione deve essere sviluppata tramite il framework **UIKit**, garantendo un'esperienza d'uso coerente con gli standard IOS.
- **Computer Vision:** L'acquisizione tramite fotocamera deve integrare **VisionKit** e il framework **Vision** per il riconoscimento **OCR** del testo.
- **Comunicazione:** L'invio delle mail di supporto deve appoggiarsi al framework **MessageUI**.
- **Compatibilità:** Il software deve essere compatibile con i dispositivi Apple (iPhone, iPad) equipaggiati con iOS 13.1 o versioni successive.

3.3 Requisiti di dominio

Il software deve operare nel rispetto dei vincoli legali imposti dal dominio di appartenenza:

Protezione dei Dati (GDPR): Trattando informazioni legate all'account iCloud dell'utente, l'applicazione deve garantire la conformità al Regolamento Generale sulla Protezione dei Dati, assicurando la trasparenza e la sicurezza nel trattamento delle informazioni personali.

3.4 Use Case Diagram

Il diagramma dei casi d'uso (Figura 1) modella le interazioni tra l'utente e il sistema, evidenziando le funzionalità principali e il coinvolgimento di framework esterni come attori secondari :

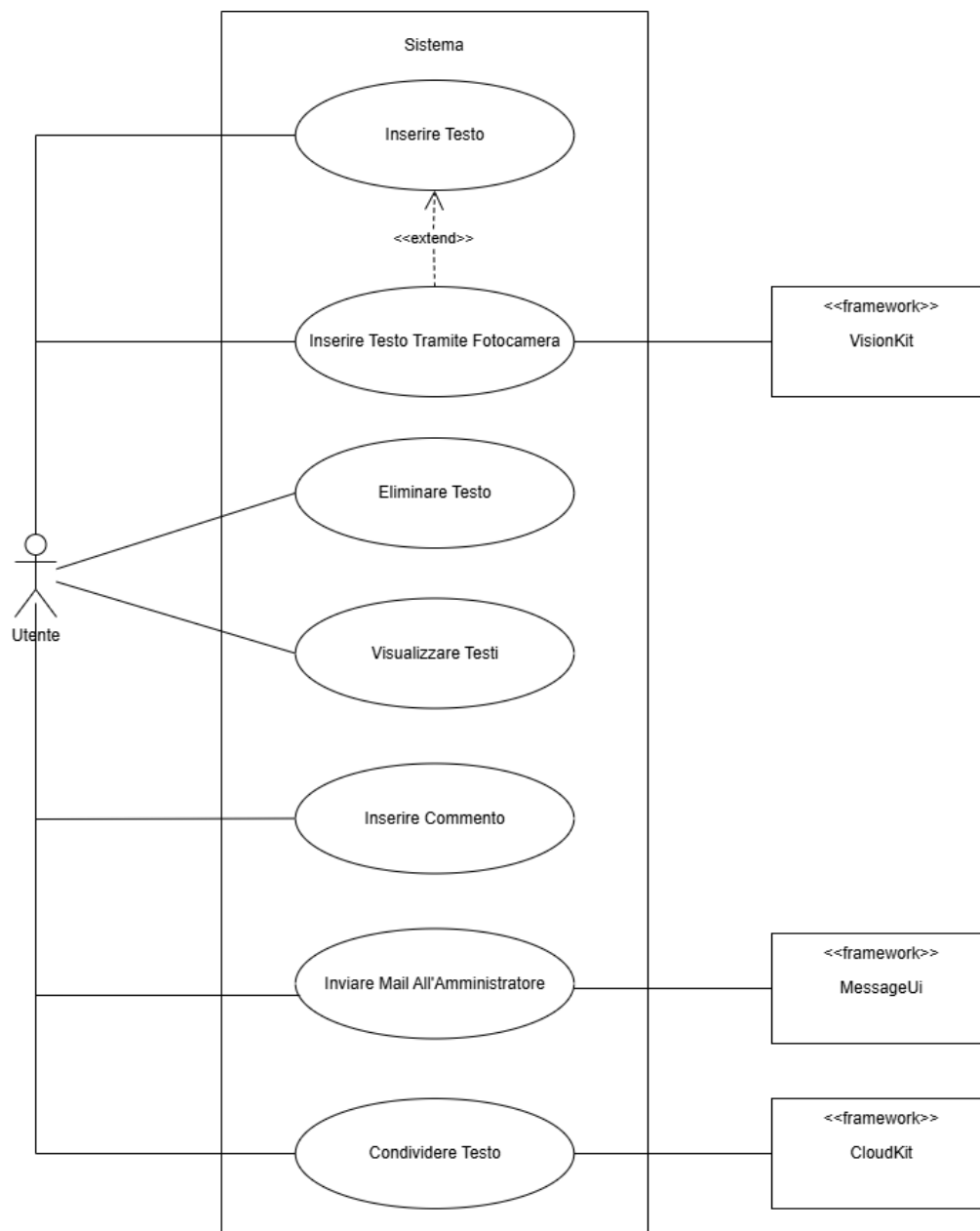


Figura 1: Use Case Diagram

3.5 Mock-Up

La progettazione dell'interfaccia è stata guidata dalla creazione di prototipi a bassa fedeltà, finalizzati a definire l'architettura informativa senza distrazioni grafiche:

Homepage

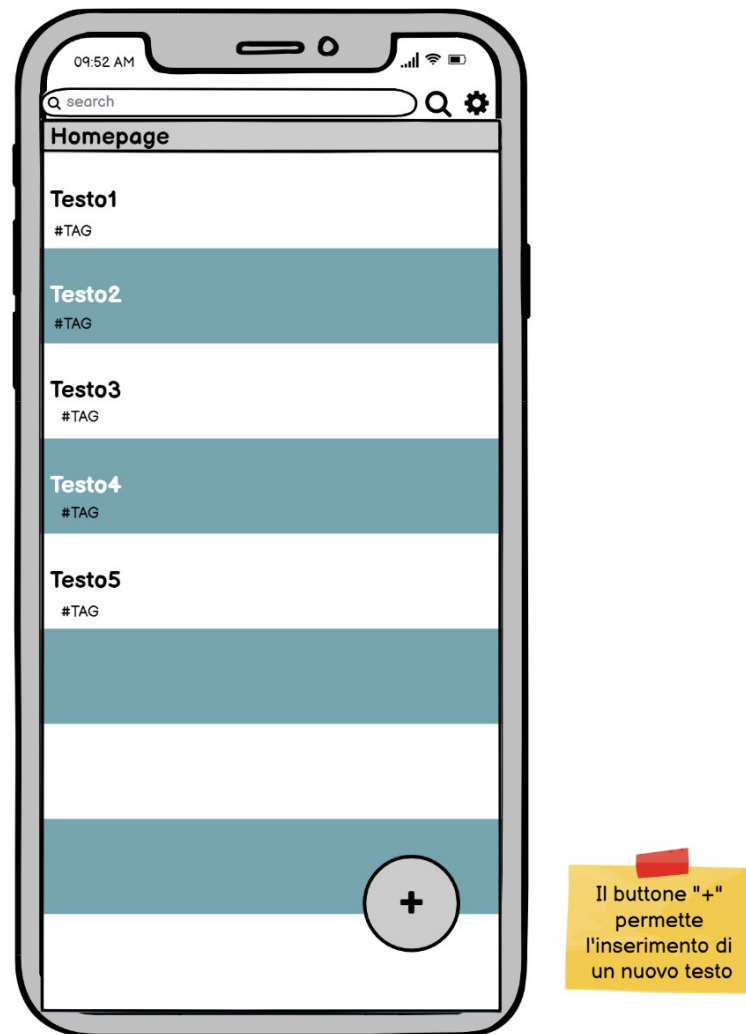


Figura 2: Mock-Up Homepage

Testo e relativi commenti

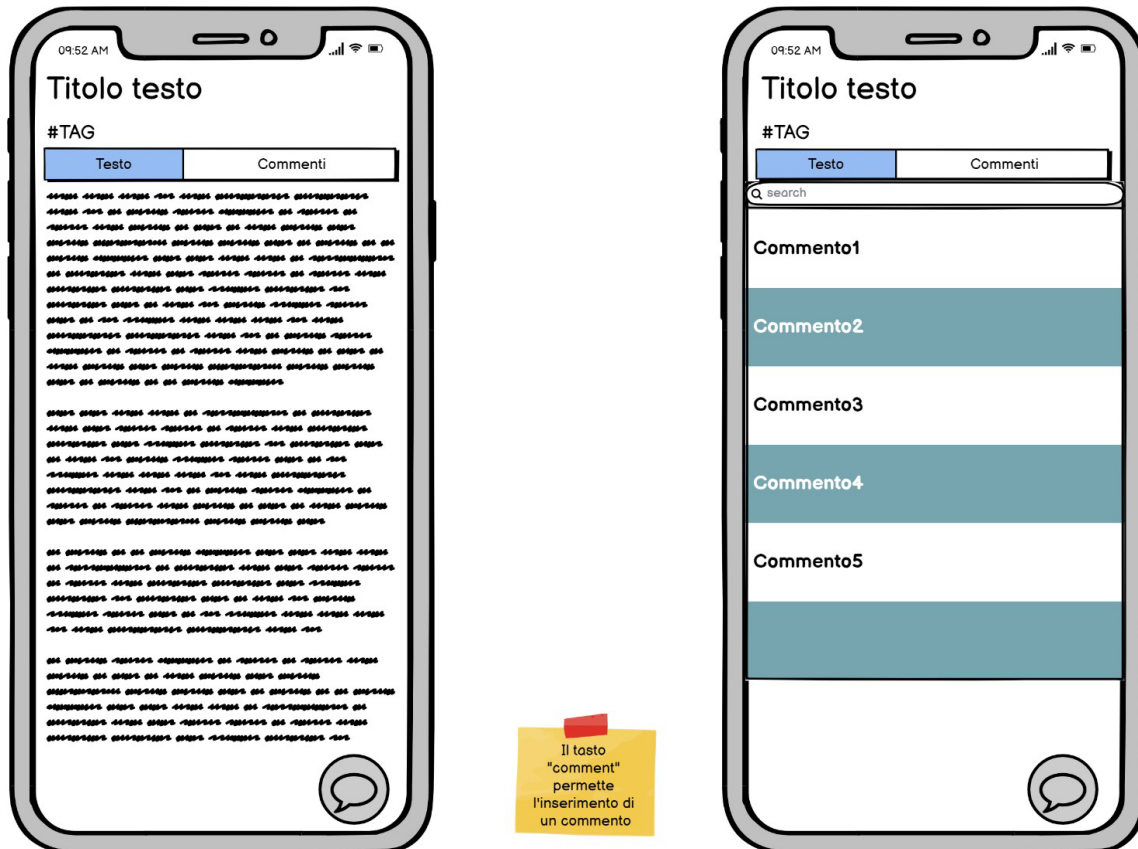


Figura 3: Mock-Up Visualizzazione testo e commenti

Aggiunta testo

The mockup shows a mobile app interface for adding text. At the top, the status bar displays '09:52 AM' and signal/battery icons. The app header includes a back arrow, the text 'YourText', and a 'Save' button. Below the header, the word 'New' is displayed. There are two input fields: 'Titolo' and '#TAG'. The main section is titled 'Testo' and contains a camera icon. Below the camera icon, there are three paragraphs of placeholder text represented by small black squares. The bottom of the screen shows a standard Android navigation bar with back, home, and recent apps buttons.

Cliccando il bottone "camera" verrà aperta la fotocamera, che permetterà la scansione del testo fotografato.

L'inserimento del testo può avvenire da tastiera o facendo un scan tramite fotocamera

Figura 4: Mock-Up Inserimento testo

Aggiunta commento



Figura 5: Mock-Up Inserimento commento

Elimina Testo / Commento

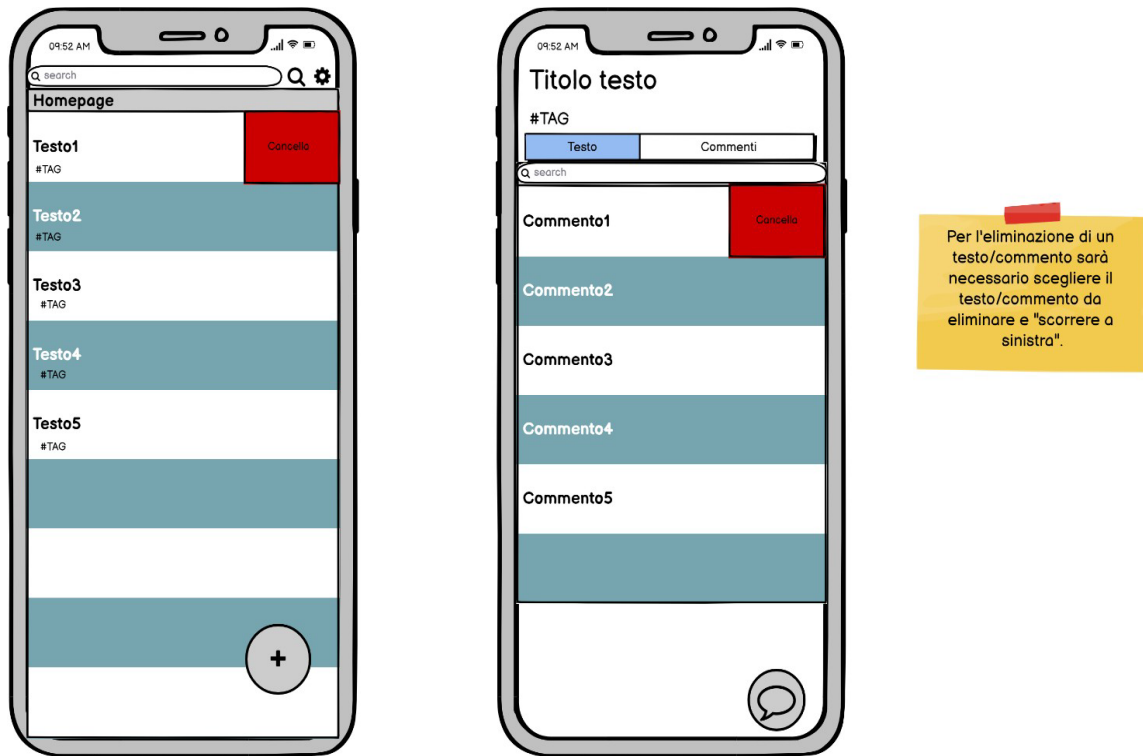


Figura 6: Eliminazione testo/commento

- **Figura 2 (Homepage):** Mostra l'elenco dei testi con l'indicatore di caricamento e il tasto "+" per l'attivazione dell'Action Sheet di inserimento.
- **Figura 3 e 4:** Illustrano il dettaglio del documento e il modulo di inserimento, dove l'utente può popolare i campi Titolo, Tag e Testo (manualmente o tramite scan).
- **Figura 5 (Commenti):** Evidenzia il meccanismo di sottolineatura del testo e l'apertura della modale per l'inserimento della nota.

3.6 Tabelle di Cockburn

Di seguito vengono riportate le specifiche dettagliate degli scenari d'uso principali, aggiornate in base al comportamento effettivo dell'applicativo:

3.6.1 Tabella di Cockburn per l'inserimento di un nuovo testo

Use Case #1	Inserisce un nuovo testo	
Primary Actor	Utente	
Trigger	L'utente fa tap sul bottone circolare "+" in basso a destra nella schermata Homepage.	
Preconditions	L'utente ha effettuato l'accesso all'applicazione e si trova nella Homepage.	
Success end condition	Il testo viene caricato correttamente sul database (CloudKit) e l'utente viene reindirizzato alla schermata di dettaglio del testo appena creato.	
Failed end condition	L'utente preme "Cancel" (o "Back") annullando l'operazione, oppure si verifica un errore di connessione durante il caricamento.	
Main Scenario		
Step #	Utente	Sistema
1	Apri l'applicazione.	Mostra la schermata Homepage (Figura 2).
2	Preme il tasto circolare "+" in basso a destra.	Mostra in basso un Action Sheet con le opzioni: "Type/Paste", "Scan", "Cancel".
3	Seleziona l'opzione "Type/Paste".	Mostra la schermata di inserimento del nuovo testo (Figura 4).
4	Compila i campi richiesti	Mostra un indicatore di

	(Titolo, #TAG, Testo) e preme "Save" in alto a destra.	caricamento a schermo con la dicitura "Uploading to iCloud".
5		Concluso il salvataggio su database, chiude la schermata di inserimento e mostra la schermata di Dettaglio del Testo appena creato.
Extension #1: L'utente non inserisce il titolo		
Step #	Utente	Sistema
4.1	Preme "save" in alto a destra ma non inserisce il titolo del testo	
5.1		Mostra un pop-up di errore chiedendo di inserire un titolo
6.1	Preme "ok"	
7.1		Torna allo step 4 del main scenario
Extension #2 L'utente preme "back" nella schermata di inserimento		
Step #	Utente	Sistema
4.2	Preme il tasto "back" in alto a sinistra	
5.2		Torna alla schermata Homepage, figura 2

Extension #3 L'utente non inserisce il "#TAG"		
Step #	Utente	Sistema
4.3	Preme "save" in alto a sinistra ma non inserisce il tag	
5.3		Mostra un pop-up di errore chiedendo di inserire un tag
6.3	Preme "ok"	
7.3		Torna allo step 4 del main scenario
Extension 4 L'utente non inserisce il testo		
Step #	Utente	Sistema
4.4	Preme "save" in alto a sinistra ma non inserisci il testo	
5.4		Mostra un pop-up di errore chiedendo di inserire il testo
6.4	Preme "ok"	
7.4		Torna allo step 4 del main scenario
# Subvariation L'utente inserisce il testo tramite fotocamera		
Step	Utente	Sistema

3.1	Nell'Action Sheet (Step 2), seleziona "Scan".	Apri l'interfaccia di sistema della fotocamera (tramite VisionKit) per scannerizzare il documento.
3.2	Scatta la foto al testo e conferma l'acquisizione.	Effettua l'OCR, converte l'immagine in stringa e mostra la schermata di inserimento (Figura 4) con il campo "Testo" già precompilato.
3.3		Il flusso riprende dallo Step 4 del Main Scenario.

Tabella 3.1: Tabella di Cockburn per lo Use Case "Inserisce un nuovo testo"

3.6.2 Tabella di Cockburn per l'aggiunta di un nuovo commento

Use Case #2	Aggiunge un nuovo commento	
Primary Actor	Utente	
Trigger	L'utente seleziona una porzione di testo e preme il bottone mobile in basso a destra (icona del fumetto).	
Preconditions	L'utente si trova nella schermata di dettaglio di un testo ed è posizionato sul tab "Testo".	
Success end condition	Il commento viene caricato su CloudKit e salvato con successo; l'utente può visualizzarlo nell'apposita sezione del testo.	
Failed end condition	L'utente preme il tasto "Back" interrompendo la creazione del commento.	
Main Scenario		
Step #	Utente	Sistema
1	Seleziona un testo dalla Homepage	Apri la schermata di dettaglio del testo, posizionandosi di default sul

		segmento "Text".
2	Tiene premuto sullo schermo per evidenziare (selezionare) la specifica porzione di testo che desidera commentare.	Mantiene evidenziato il testo selezionato dall'utente.
3	Preme il bottone "aggiungi commento" (icona a fumetto) in basso a destra.	Mostra la schermata "Nuovo commento" (Figura 5), pre-compilando automaticamente il box superiore con il testo appena evidenziato.
4	Digita il proprio dubbio/spiegazione nel campo di testo inferiore e preme "Done" sulla tastiera.	Mostra un indicatore di caricamento a schermo con la dicitura "Uploading to iCloud".
5		Al termine del salvataggio, chiude la modale e riporta l'utente alla schermata di dettaglio del testo, rendendo il commento disponibile nel tab "Comments".
Extension #1: L'utente tenta di commentare senza aver selezionato il testo		
Step #	Utente	Sistema
3.1	Tenta di premere il tasto di creazione commento senza aver prima evidenziato una porzione di testo.	Il sistema non permette l'apertura della schermata di inserimento o mostra un pop-up di errore chiedendo di sottolineare una porzione di

		testo.
3.2	Preme "OK" sul pop-up.	Riporta l'utente allo Step 2 del Main Scenario.
Extension #2 L'utente preme "Back"		
Step #	Utente	Sistema
4.1	Preme il tasto "back" in alto a sinistra	Annulla l'operazione e riporta l'utente alla schermata di dettaglio del testo senza salvare nulla.
Extension #3 L'utente non inserisce il testo del commento		
Step #	Utente	Sistema
4.2	Tenta di salvare premendo "Done" o "Aggiungi" lasciando vuoto il campo del commento.	Mostra un pop-up di errore segnalando che è necessario inserire un testo per il commento.
4.3	Preme "OK" sul pop-up.	Mostra nuovamente la schermata di inserimento per permettere la correzione.

Tabella 3.2: Tabella di Cockburn per lo Use Case "Inserisce un nuovo commento"

3.6.3 Tabella di Cockburn per l'eliminazione di un testo

Use Case #3	Elimina un testo e i relativi commenti
Primary Actor	Utente
Trigger	L'utente effettua uno "swipe" verso sinistra sulla cella di un testo e preme il bottone rosso "Cancella".
Preconditions	L'utente si trova nella Homepage e ha almeno un testo salvato nel database.

Success end condition	Il testo e tutti i commenti ad esso associati vengono eliminati definitivamente (eliminazione a cascata) e la riga scompare dalla UI.	
Failed end condition	L'utente non conferma l'eliminazione nel pop-up di avvertimento.	
Main Scenario		
Step #	Utente	Sistema
1	Effettua uno swipe (scorrimento) verso sinistra sulla riga del testo che desidera eliminare.	Rivela un bottone rosso con la dicitura "Cancella" (Figura 6) sul lato destro della cella.
2	Preme il tasto rosso "Cancella".	Mostra un pop-up di avvertimento per chiedere la conferma dell'eliminazione (es. "Sei sicuro di voler eliminare questo testo?").
3	Preme "Sì" (o "Conferma") sul pop-up.	Avvia l'eliminazione a cascata sul database (prima i commenti figli, poi il testo padre) e rimuove la cella dalla tabella aggiornando la Homepage.
Extension #1: L'utente non conferma la cancellazione		
Step #	Utente	Sistema
3.1	Preme "No" (o "Annulla") sul pop-up di avvertimento.	Chiude il pop-up, nasconde il bottone rosso "Cancella" e ripristina la schermata Homepage senza effettuare alcuna operazione sul

		database.
--	--	-----------

Tabella 3.3: Tabella di Cockburn per lo Use Case "Elimina un testo"

La tabella di Cockburn per il caso d'uso di eliminazione di un commento è stata omessa, in quanto è analoga alla cancellazione di un testo (Tabella 3.3).

3.6.4 Tabella di Cockburn per la condivisione di un testo

Use Case #4	Condivide un testo con altri utenti	
Primary Actor	Utente	
Trigger	L'utente fa tap sul tasto "Share" (icona omino con il +) nella navigation bar del MasterViewController.	
Preconditions	L'utente ha effettuato l'accesso a iCloud e si trova nella visualizzazione di un testo esistente.	
Success end condition	Viene generato un link di partecipazione e inviato correttamente; il testo appare nel database condiviso del destinatario.	
Main Scenario		
Step #	Utente	Sistema
1	Preme il tasto di condivisione.	
2		Inizializza un oggetto CKShare e apre l'interfaccia nativa UICloudSharingController.
3	Seleziona il metodo di	

	invio (es. WhatsApp, Mail, iMessage) e il destinatario.	
4		Invia l'invito crittografato.
Extension #1: L'utente annulla l'operazione		
Step #	Utente	Sistema
3.1	Non sceglie nessun modo per condividere il testo.	Chiude l'interfaccia di condivisione senza creare il record di sharing.

Tabella 3.4: Tabella di Cockburn per lo Use Case “Condivide un testo”

3.7 Class Diagram del database

Il modello logico dei dati (Figura 7) descrive la struttura delle informazioni persistite su CloudKit.

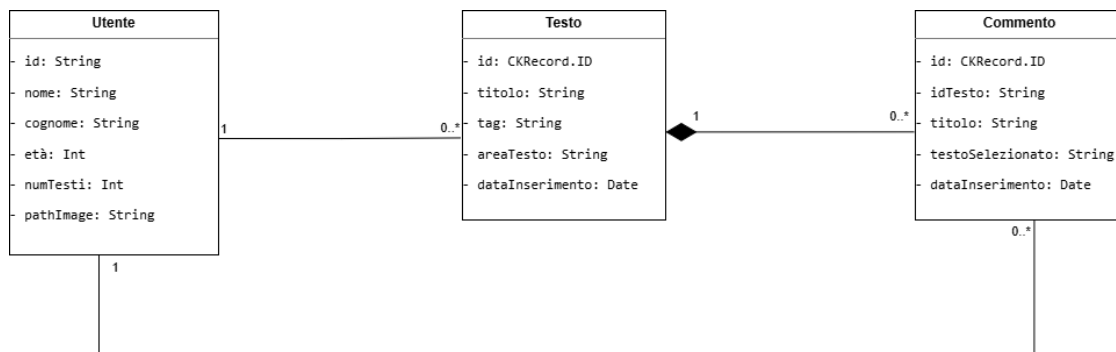


Figura 7: Class diagram del database

Analisi del Modello: Il diagramma evidenzia tre entità principali. La classe **Testo** è legata alla classe **Commento** da una relazione di **composizione** (rombo pieno), indicando che il ciclo di vita del commento è strettamente dipendente da quello del testo padre. Entrambe

le classi integrano un attributo id di tipo CKRecord.ID per la gestione univoca sul cloud. L'entità **Utente** funge da proprietario delle risorse, mantenendo riferimenti ai testi creati e ai commenti inseriti.

Capitolo 4

Software design

In questo capitolo vengono esplicitate le scelte progettuali relative all'architettura del software a diversi livelli di astrazione. Partendo dai design pattern adottati per garantire modularità e scalabilità, la trattazione approfondisce il System Design attraverso il modello **Model-View-Controller (MVC)** nella sua implementazione specifica per l'ambiente Swift. Infine, l'Object Design dettaglia il comportamento dinamico delle funzionalità core tramite diagrammi di sequenza, di attività e diagrammi di stato, correlandoli ai frammenti di codice più significativi

4.1 Design Pattern

Un design pattern rappresenta una soluzione consolidata a problemi ricorrenti di progettazione software. L'adozione di tali schemi permette di ottenere un codice più leggibile e facilmente estensibile. Nell'applicativo in esame sono stati impiegati i seguenti pattern:

- **Composite:** Utilizzato per gestire la gerarchia delle viste all'interno dell'interfaccia utente.
- **Strategy:** Consente di isolare e variare gli algoritmi di gestione degli input (es. inserimento manuale vs scanner).
- **Observer:** Fondamentale per la notifica dei cambiamenti di stato dal modello alla vista, implementato nativamente tramite il meccanismo delle notifiche e delle callback.
- **Mediator:** Il Controller agisce da mediatore per disaccoppiare la logica di business dalla presentazione.

- **Data Access Object (DAO):** Impiegato per astrarre le operazioni di persistenza su CloudKit, separando la logica dei dati dal resto del sistema.
- **Singleton:** Garantisce un punto di accesso globale unico per le istanze dei controller e del database.

4.2 System Design : L'architettura MVC in Swift

L'architettura globale del sistema è stata progettata seguendo il pattern architetturale Model-View-Controller (MVC). Questo paradigma ha lo scopo di separare la rappresentazione delle informazioni (View) dalla logica di business e dai dati (Model), utilizzando un intermediario (Controller) per gestire la comunicazione tra le parti [4].

Nella sua formulazione teorica classica (Figura 8), il pattern MVC prevede che la View possa richiedere lo stato direttamente al Model e che quest'ultimo possa notificare i propri cambiamenti alla View. Tuttavia, questa stretta dipendenza tra Vista e Modello tende a ridurre la riusabilità del codice e a creare un forte accoppiamento (coupling) tra componenti che dovrebbero rimanere indipendenti.

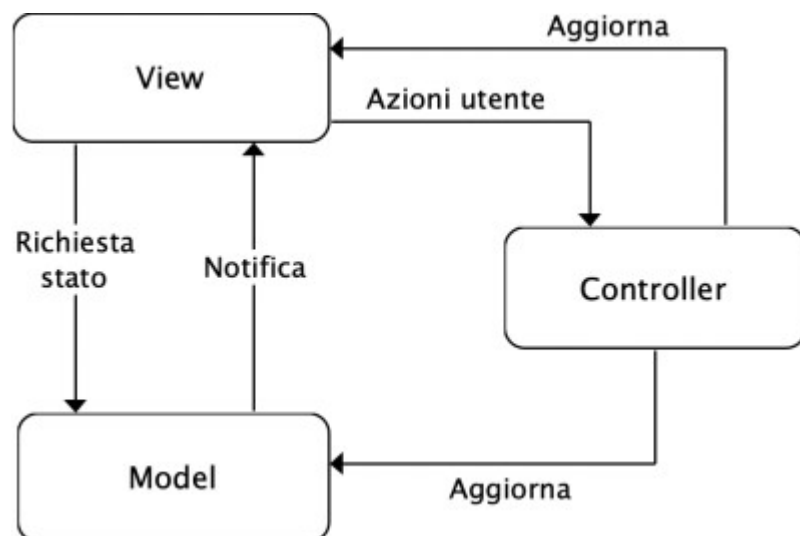


Figura 8:Modello MVC

Per risolvere questa criticità e favorire una maggiore modularità, in ambiente iOS e nel linguaggio Swift si adotta una variante specifica del pattern MVC (Figura 9). In questa implementazione, la View e il Model sono completamente disaccoppiati e non comunicano mai in modo diretto.

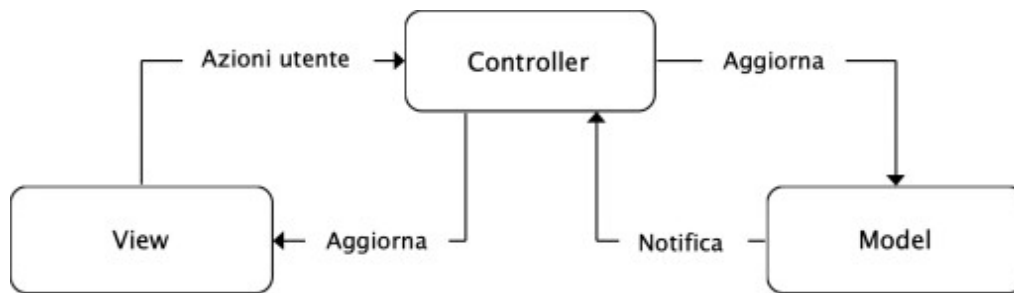


Figura 9: Modello MVC in Swift

Come si evince dal diagramma, il Controller assume un ruolo centrale e indispensabile:

1. **View to Controller:** Le azioni dell'utente sull'interfaccia (es. tap su un pulsante, swipe) vengono catturate dalla View e inviate al Controller tramite il pattern Command (meccanismo target-action) o Delegate.
2. **Controller to Model:** Il Controller elabora l'input, applica la logica di business (avvalendosi del pattern Strategy) e richiede al Model di aggiornare i propri dati.
3. **Model to Controller:** Quando il Model subisce una variazione di stato, notifica il Controller (utilizzando il pattern Observer, come ad esempio il NotificationCenter o le completion handler).
4. **Controller to View:** Infine, il Controller istruisce la View su come aggiornare l'interfaccia grafica per riflettere i nuovi dati.

In aggiunta a questa struttura base, l'architettura dell'applicativo è stata ulteriormente rafforzata introducendo il pattern **Data Access Object (DAO)**. Invece di far comunicare il Controller direttamente con il database (Model), il Controller si interfaccia con il DAO (es. TestoDAO), il quale incapsula tutta la complessa logica di persistenza e

sincronizzazione con **CloudKit**. Questo livello di astrazione aggiuntivo garantisce un codice estremamente pulito e facilita le future operazioni di manutenzione.

4.3 Object Design

Con **Object Design** si intende la fase di progettazione di basso livello, in cui l'architettura generale viene declinata nella definizione specifica delle classi, dei loro metodi e delle reciproche interazioni. Lo scopo di questa fase è dettagliare il comportamento statico e dinamico delle singole funzionalità previste dal software, avvalendosi dei diagrammi UML (**Class Diagram**, **Sequence Diagram**, **Activity Diagram** e **State Chart**).

4.3.1 Architettura Statica: Class Diagram Globale

Per avere una visione d'insieme della struttura dell'applicativo, è stato realizzato il Class Diagram globale del sistema.

Il diagramma evidenzia la rigida separazione dei ruoli imposta dal pattern MVC e dal pattern DAO:

- **View Controllers:** Classi come ViewController, AddTraductionViewController e CommentViewController si occupano esclusivamente di gestire l'interfaccia utente (UI) e catturare gli input.
- **Controllers di Business:** Per evitare il sovraffollamento dei View Controller, la logica di dominio è stata estratta in classi dedicate, come InserimentoController e CancellazioneController.
- **Data Access Object:** La classe TestoDAO rappresenta l'unico punto di accesso al database remoto. Essa detiene l'istanza del privateCloudDatabase di CloudKit ed espone metodi standardizzati (salva(), recuperaTutti(), cancella()) per manipolare le entità del modello (Testo e Commento).

4.3.2 Flusso di Inserimento e Scansione (Activity e State Chart)

L'inserimento di un nuovo testo è un'operazione che offre all'utente due percorsi alternativi: l'inserimento manuale o l'acquisizione tramite scanner intelligente. Questa biforcazione logica è modellata nel seguente Activity Diagram.

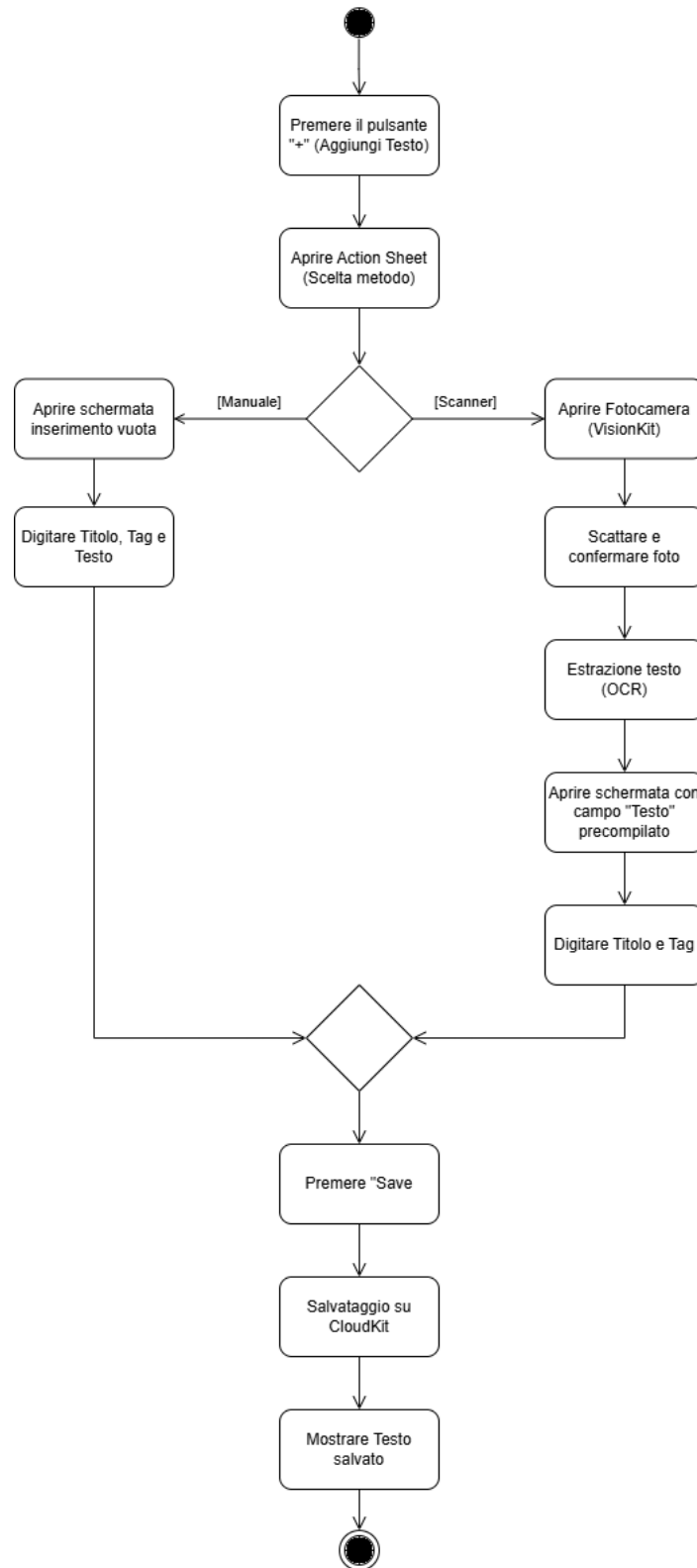


Figura 11: Activity Diagram per la funzionalità di inserimento testo

Come illustrato, l'utente sceglie il metodo tramite un Action Sheet. Se opta per lo scanner, il sistema avvia un flusso più complesso che prevede l'acquisizione dell'immagine e l'estrazione del testo (OCR).

Per dettagliare il ciclo di vita dell'oggetto durante questa specifica operazione di scansione, è stato elaborato uno **State Chart Diagram**.

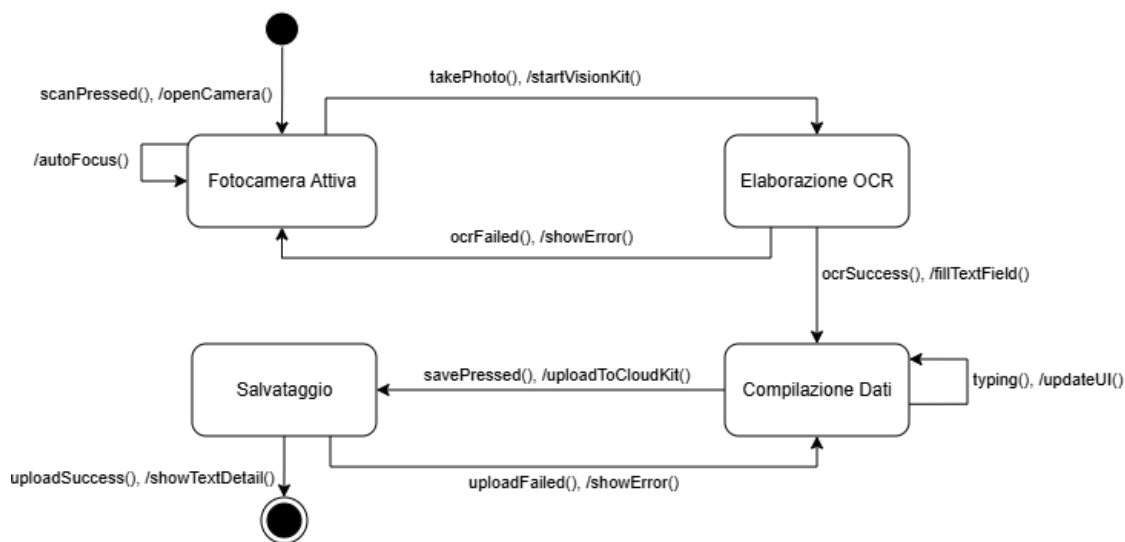


Figura 12: State Chart Diagram per il processo di scansione e salvataggio

Il diagramma degli stati mostra le fasi interne dell'applicativo: dalla "Fotocamera Attiva" si passa all'"Elaborazione OCR" tramite VisionKit. In caso di successo (ocrSuccess()), il sistema transita nello stato di "Compilazione Dati", permettendo all'utente di digitare Titolo e Tag. Infine, la pressione del tasto di salvataggio innesca lo stato di "Salvataggio", con gestione esplicita delle eccezioni (es. uploadFailed()) per garantire la robustezza del software.

4.3.3 Dinamica dell'Acquisizione OCR (Sequence Diagram)

L'interazione tra i componenti di sistema durante l'attivazione della fotocamera e l'estrazione del testo è descritta dal seguente diagramma di sequenza.

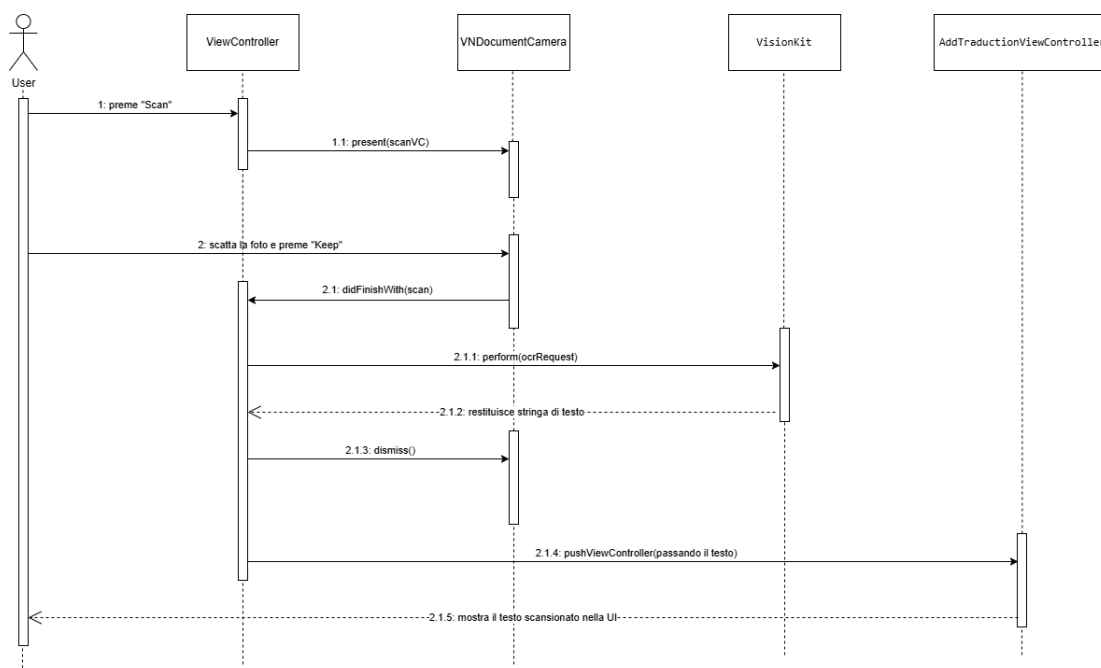


Figura 13: Sequence Diagram del flusso OCR tramite VisionKit

Il **ViewController** invoca il **VNDocumentCameraViewController** di sistema. Una volta che l'utente scatta la foto, l'immagine viene passata a **VisionKit** che, tramite una **VNRecognizeTextRequest**, restituisce la stringa di testo elaborata. Questa stringa viene infine iniettata nel **AddTraductionViewController**, precompilando la View per l'utente.

4.3.4 Inserimento di un Nuovo Testo su CloudKit

Una volta compilati i dati, il processo di persistenza sul database cloud segue un flusso rigoroso per disaccoppiare l'interfaccia dalla rete.

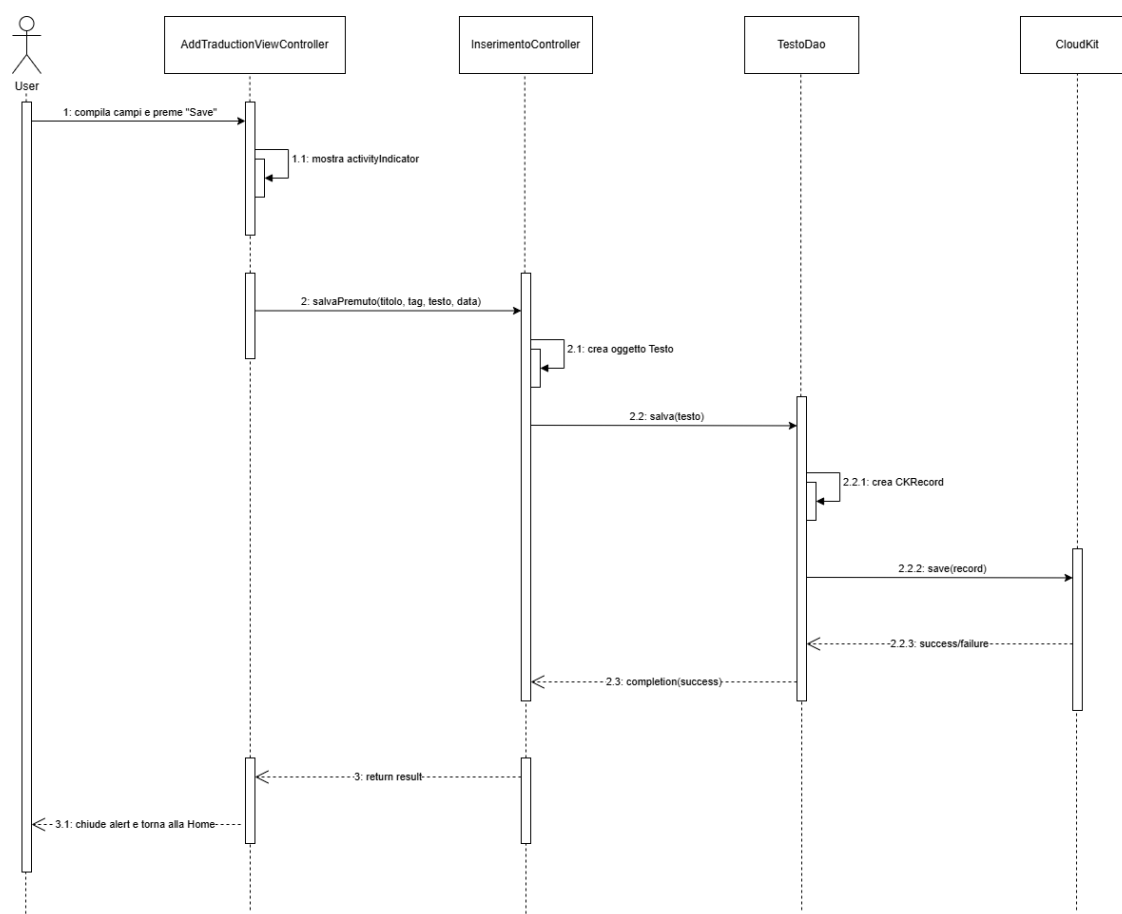


Figura 14: Sequence Diagram per il salvataggio di un nuovo testo

L'**AddTraductionViewController** demanda l'operazione all'**InserimentoController**, il quale istanzia l'oggetto **Testo** e lo passa al **TestoDAO**. Il DAO si occupa di tradurre l'oggetto in un **CKRecord** e di effettuare la chiamata asincrona **save(record)** verso i server di **CloudKit**. Il risultato (successo o fallimento) viene propagato a ritroso fino alla View tramite callback.

4.3.5 Inserimento di un Commento Semantico

In modo del tutto analogo al testo principale, l'inserimento di un commento su una porzione di testo evidenziata sfrutta l'architettura a livelli per garantire la consistenza dei dati.

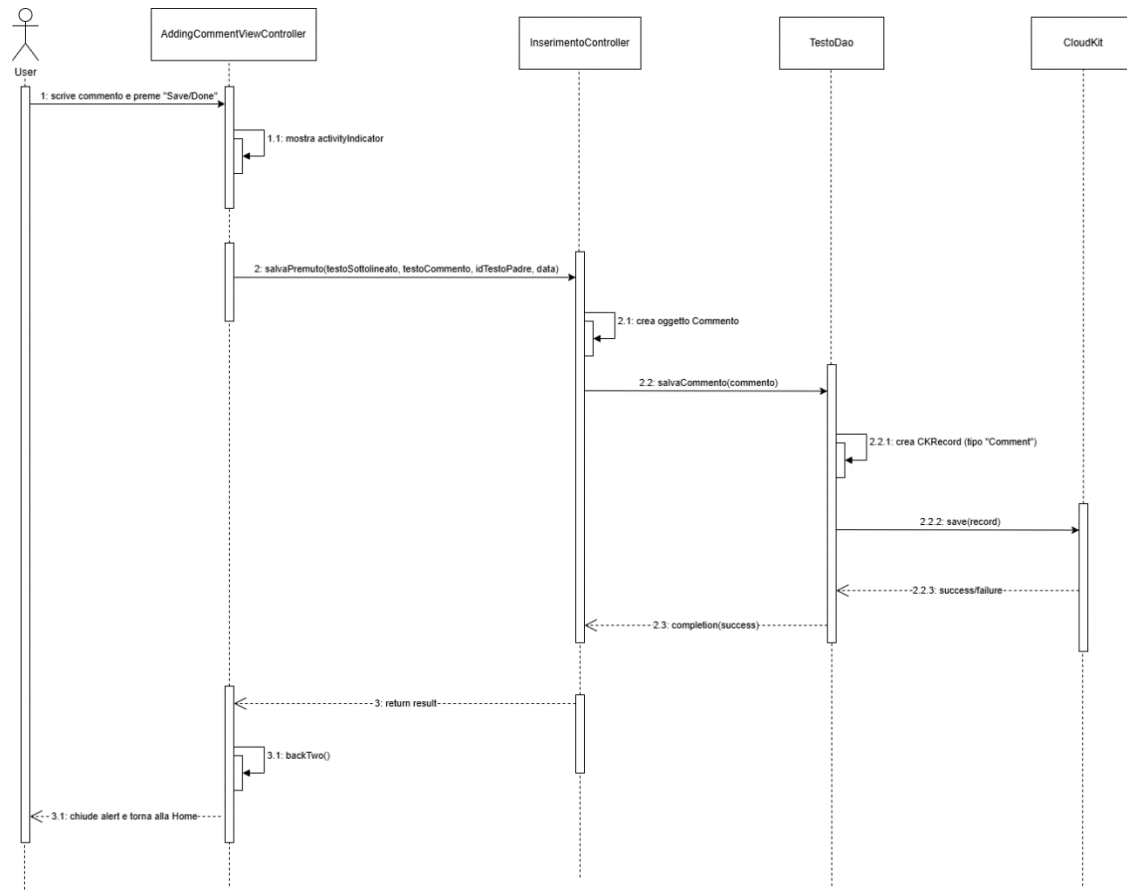


Figura 15: Sequence Diagram per l'inserimento di un commento

L'**AddingCommentViewController** raccoglie la stringa evidenziata e il testo digitato, invocando il metodo dedicato nell'**InserimentoController**. Quest'ultimo crea un oggetto **Commento** associandogli l'**idTesto** (la chiave esterna che lo lega al testo padre) e lo inoltra al DAO per il salvataggio su CloudKit.

4.3.6 Sincronizzazione e Visualizzazione dei Testi

Per garantire che l'utente visualizzi sempre i dati aggiornati all'apertura dell'applicazione, è stato implementato un flusso di recupero dati asincrono.

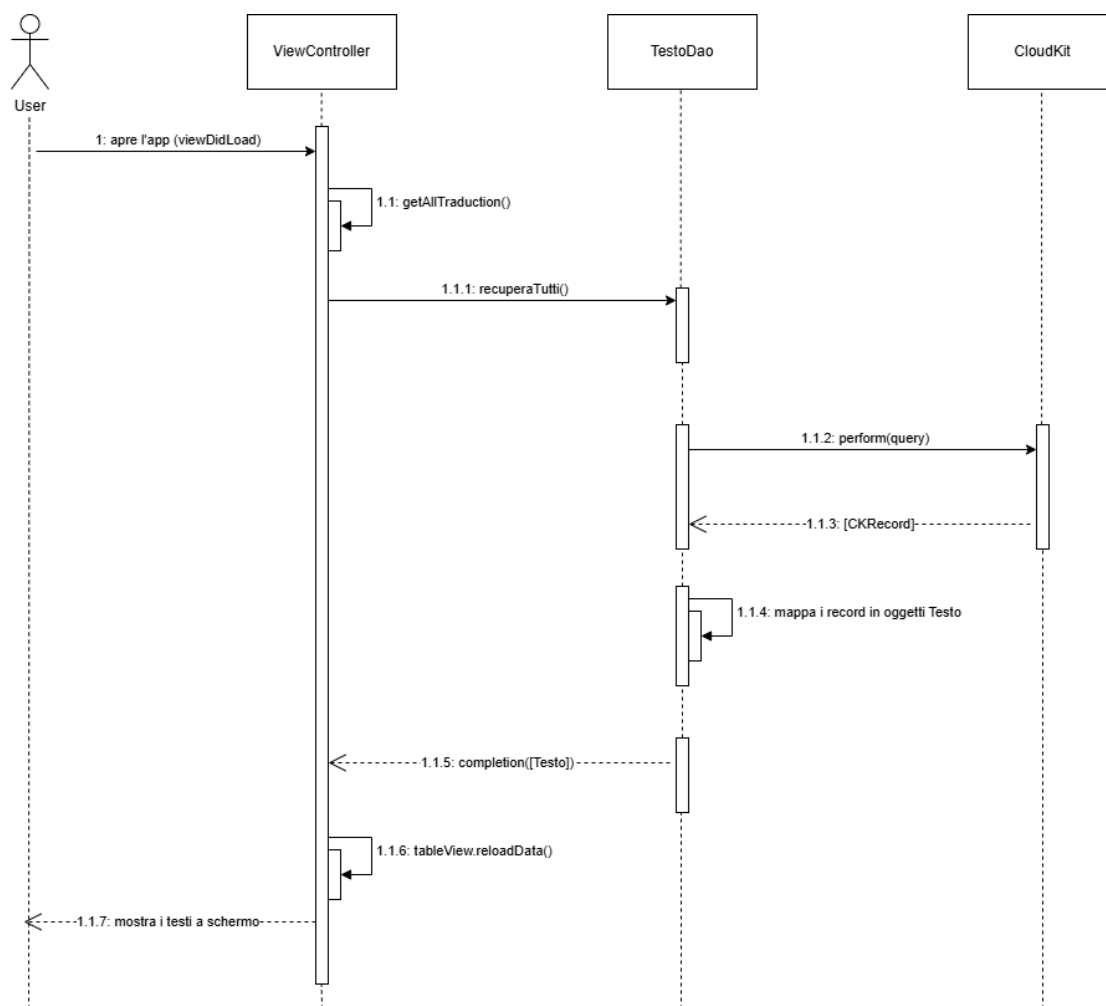


Figura 16: Sequence Diagram per il recupero dei dati da CloudKit

All'avvio (**viewDidLoad**), il **ViewController** richiama il **TestoDAO** per ottenere l'elenco dei testi. Il DAO esegue una **CKQuery** su CloudKit, ordina i record per data di creazione e mappa i **CKRecord** scaricati in un array di oggetti Testo, restituendoli al Controller per l'aggiornamento della **UITableView**.

4.3.7 Eliminazione a Cascata di un Testo

La rimozione di un documento testuale richiede un'attenzione particolare, in quanto l'eliminazione del record padre deve implicare la rimozione logica di tutti i commenti figli associati, per evitare inconsistenze nel database.

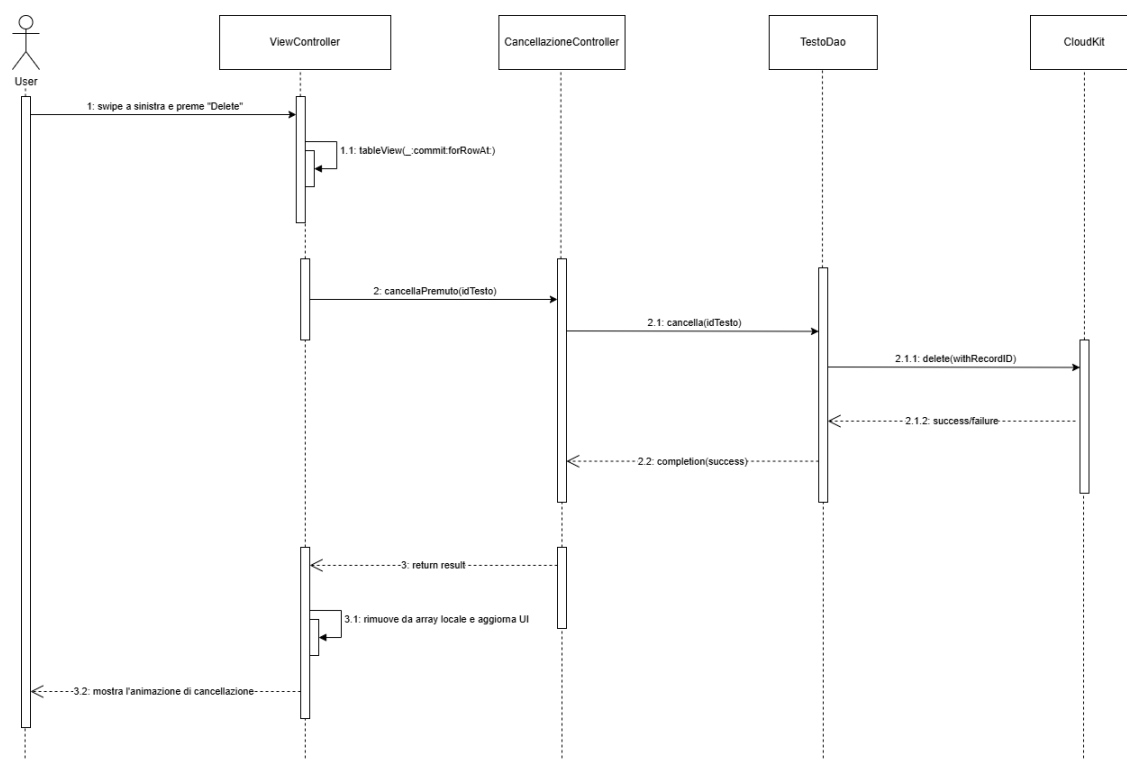


Figura 17: Sequence Diagram per la funzionalità di Eliminazione a Cascata

Quando l'utente effettua lo swipe per cancellare, il **ViewController** applica un approccio Optimistic UI, rimuovendo immediatamente l'elemento dall'interfaccia per garantire fluidità. Parallelamente, delega la cancellazione fisica al **CancellazioneController**, che instruisce il **TestoDAO**. Nel DAO, come delineato nella logica di business, l'eliminazione del **CKRecord** padre tramite il suo **id** viene processata sul database remoto.

Capitolo 5

Testing e piano di testing

Il presente capitolo descrive la fase conclusiva del ciclo di vita del software: il testing. Questa fase risulta essenziale per rilevare anomalie, risolvere bug e garantire l'affidabilità del sistema prima del rilascio. La trattazione si divide in due sezioni principali: la fase di verifica strutturale del codice, condotta tramite Unit Testing automatizzati, e la fase di validazione del software, effettuata sul campo tramite il programma di beta testing offerto dalla piattaforma Apple TestFlight.

5.1 Unit Testing

L'Unit Testing è una pratica di ingegneria del software che consiste nel testare individualmente le componenti più piccole e isolate di un programma (i metodi o le funzioni). Lo scopo è verificare che la logica interna del codice produca esattamente l'output atteso e rispetti quanto modellato nei Sequence Diagram.

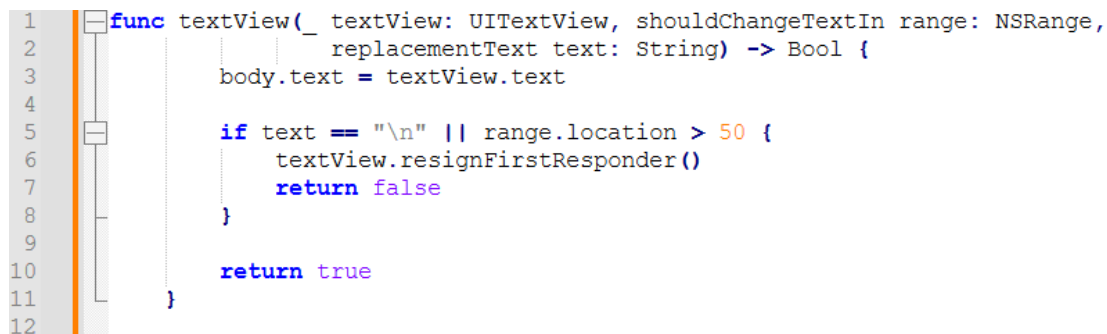
Nell'ambiente di sviluppo Xcode, i test sono stati implementati sfruttando il framework nativo XCTest. Per garantire una copertura ottimale, le unità sono state analizzate combinando due macro-strategie complementari:

- **Strategia Black Box:** Basata esclusivamente sui requisiti funzionali dell'unità, analizzando gli input e gli output senza curarsi dell'implementazione interna.
- **Strategia White Box:** Basata sulla struttura e sui rami decisionali del codice sorgente, assicurandosi che ogni percorso del Grafo del Flusso di Controllo (GFC) venga eseguito almeno una volta.

5.1.1 Testing del metodo di validazione input (SendMailViewController)

Un caso di studio significativo per la fase di Unit Testing riguarda il controllo dell'input utente durante la segnalazione di un bug (Use Case "Inviare Mail All'Amministratore"). Nel SendMailViewController, la digitazione all'interno della view testuale è controllata dal metodo delegato `textView(_:shouldChangeTextIn:replacementText:)`.

Tale metodo ha il compito di restituire un valore booleano (true in caso di input ammesso, false in caso di inserimento bloccato). Un inserimento è considerato valido se non contiene ritorni a capo (`\n`) e non supera la lunghezza massima consentita di 50 caratteri.



```
1 func textView(_ textView: UITextView, shouldChangeTextIn range: NSRange,
2               replacementText text: String) -> Bool {
3     body.text = textView.text
4
5     if text == "\n" || range.location > 50 {
6         textView.resignFirstResponder()
7         return false
8     }
9
10    return true
11 }
12
```

Figura 18: Codice del metodo `textView`

Per testare questo metodo con la tecnica **Black Box** (Equivalence Partitioning), sono state definite le seguenti classi di equivalenza per i parametri in ingresso:

- **CE1**: Inserimento di un ritorno a capo `\n` (Classe **Non Valida**, si aspetta false).
- **CE2**: Inserimento di un testo entro il limite dei 50 caratteri, ad esempio lunghezza 5 (Classe **Valida**, si aspetta true).
- **CE3**: Inserimento in un range superiore a 50, ad esempio alla posizione 51 (Classe **Non Valida**, si aspetta false).

Applicando la strategia *Weak Equivalence Class Testing*, sono stati implementati all'interno della suite tre test unitari automatizzati, uno per ogni classe:


```

3 // Test 1: CE3 (Non Valida) - Inserimento oltre i 50 caratteri
4 func testRangeFalse() throws {
5     // Simuliamo un range maggiore di 50 (es. 51)
6     let range = NSRange(location: 51, length: 0)
7     let stringaLunga = "prova6prova6prova6prova6prova6prova6prova6prova6"
8
9     // Chiamiamo il metodo del delegate (che nella tesi chiami checktextView)
10    let value = sut.textView(textViewTest, shouldChangeTextIn: range, replacementText: stringaLunga)
11
12    // Verifichiamo che restituisca FALSE
13    XCTAssertFalse(value, "Expected false to be passed to the assert")
14 }
15
16 // Test 2: CE2 (Valida) - Inserimento sotto i 50 caratteri
17 func testRangeTrue() throws {
18     // Simuliamo un range valido (es. 5)
19     let range = NSRange(location: 5, length: 0)
20
21     // Chiamiamo il metodo
22     let value = sut.textView(textViewTest, shouldChangeTextIn: range, replacementText: "prova")
23
24     // Verifichiamo che restituisca TRUE
25     XCTAssertTrue(value, "Expected true to be passed to the assert")
26 }
27
28 // Test 3: CE1 (Non Valida) - Inserimento di un "A capo" (\n)
29 func testTextFalse() throws {
30     // Simuliamo l'inserimento di un invio (ritorno a capo)
31     let range = NSRange(location: 1, length: 0)
32
33     // Chiamiamo il metodo
34     let value = sut.textView(textViewTest, shouldChangeTextIn: range, replacementText: "\n")
35
36     // Verifichiamo che restituisca FALSE
37     XCTAssertFalse(value, "Expected false to be passed to the assert")
38 }

```

Figura 19: Codice dei test

Per completare il testing in **White Box**, si fa riferimento al **Grafo del Flusso di Controllo (GFC)** dell'algoritmo.

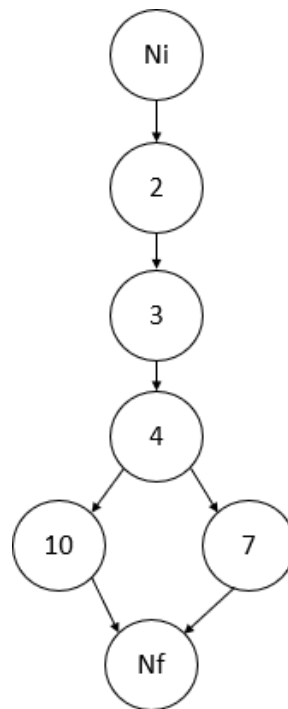


Figura 20: GFC del metodo textView

Come si evince dall'analisi del grafo, le tre classi di equivalenza modellate per il Black Box ricoprono esaustivamente tutti i possibili percorsi d'esecuzione del codice, garantendo una copertura del 100% (Branch Coverage).

5.2 TestFlight Beta Testing e Validazione UX

Il processo di validazione del software sul campo è stato condotto sfruttando **Apple TestFlight**, la piattaforma nativa per la distribuzione di versioni pre-release. Il testing è stato strutturato in due fasi incrementali e ha coinvolto sia tester interni sia un bacino di oltre cento tester esterni, costituito principalmente da studenti universitari e colleghi della Apple Developer Academy.

L'approccio iterativo ha permesso di raccogliere feedback sull'usabilità e report di crash, portando a diverse correzioni prima del consolidamento della versione finale (Release Candidate).

5.2.1 Fase 1: Prima Iterazione e Bug Fixing

La prima build dell'applicazione è stata distribuita a un campione di 52 tester. Durante questa fase sono emerse alcune criticità tecniche e limitazioni dal punto di vista della *User Experience* (UX):

- **Sincronizzazione della UI:** I tester hanno segnalato che, a seguito dell'inserimento di un nuovo testo o di un commento su CloudKit, l'interfaccia (UITableView) non si aggiornava in tempo reale, richiedendo un riavvio o un *refresh* manuale della pagina.
- **Conflitti di Eliminazione (Crash):** L'analisi dei log ha evidenziato la terminazione anomala dell'app (crash) durante il tentativo di eliminazione di un testo padre che conteneva commenti figli, a causa di conflitti nella gestione asincrona della "cancellazione a cascata".
- **Miglioramenti UX/UI:** Su suggerimento dei tester, l'interfaccia è stata arricchita. È stata implementata una *Search Bar* per filtrare i documenti tramite Titolo e Tag (non presente nella prima iterazione), sono stati inseriti degli *Empty States* (grafiche per schermate prive di dati) per la home e i commenti, ed è stato aggiunto un tutorial di onboarding iniziale.

Le anomalie tecniche sono state tempestivamente corrette, riallineando l'architettura MVC per garantire la reattività dell'interfaccia e mettendo in sicurezza le operazioni di database.

5.2.2 Fase 2: Distribuzione su Larga Scala e Telemetria

Una volta stabilizzato il codice, una seconda versione (v2.0) è stata rilasciata a un campione esteso di 108 utenti. Essendo un campione in target (studenti), è stato richiesto loro di eseguire task specifici per "stressare" le funzionalità core: inserimento testi tramite

tastiera e OCR, creazione di note semantiche, eliminazione e condivisione gerarchica dei dati.

Per documentare rigorosamente questa fase, ci si è avvalsi degli strumenti di telemetria integrati in Apple TestFlight. I dati raccolti (riassunti nella Tabella 5.1) hanno confermato il successo dell'iterazione precedente, mostrando una netta diminuzione dei faults di sistema e una percentuale di Crash-Free Sessions (sessioni prive di interruzioni) del 98.2%, denotando un'elevata stabilità dell'applicativo.

Metrica di Beta Testing (TestFlight)	Valore Registrato
Tester Esterni Invitati	108
Sessioni totali stimate	~450
Crash-Free Sessions (Stabilità)	98.2%
Versioni (Build) rilasciate	2

Tabella 5.1: Dati riassuntivi della telemetria di TestFlight

5.2.3 Valutazione dei Risultati e Sondaggio UX

Al termine del periodo di prova, al fine di misurare l'usabilità del sistema (UX) con rigore metodologico, ai tester è stato sottoposto un questionario di valutazione asincrono tramite piattaforma digitale (Google Forms).

Metodologia del Questionario

Il questionario è stato progettato includendo un totale di 8 quesiti, suddivisi in tre tipologie per garantire una raccolta dati sia quantitativa che qualitativa:

1. **Domande di profilazione (2):** Quesiti a risposta chiusa binaria (Sì/No) per verificare il target di utenza (studenti universitari) e l'utilità generale.
2. **Domande di misurazione UX (3):** Quesiti valutativi basati su una scala Likert a 5 punti (dove 1 rappresenta "Per niente" e 5 rappresenta "Moltissimo"), mirati a quantificare l'esperienza d'uso sulle funzionalità core.

3. **Domande qualitative aperte (3):** Spazi a testo libero per la segnalazione di bug, apprezzamenti e richieste di funzionalità future (*Feature Requests*).

Sono state raccolte 82 risposte valide. Il 91% del campione (75/82) ha confermato il proprio status di studente. Di questo sottogruppo, il 93% (70/75) ha dichiarato che l'applicazione risulta concretamente utile per il proprio metodo di studio.

Dettagli Statistici: Media, Varianza e Deviazione Standard

Per analizzare i risultati della scala Likert non ci si è limitati al calcolo della media aritmetica (μ), ma sono state calcolate la varianza (σ^2) e la deviazione standard (σ) per misurare la dispersione dei voti. Una deviazione standard bassa indica un elevato grado di consenso tra i tester:

- **Intuitività dell'interfaccia (Homepage):**
 - Media: $\mu = 4.2$
 - Varianza: $\sigma^2 = 0.61$
 - Deviazione Standard: $\sigma = 0.78$
- **Facilità di interazione (Sottolineatura e commento):**
 - Media: $\mu = 4.4$
 - Varianza: $\sigma^2 = 0.42$
 - Deviazione Standard: $\sigma = 0.65$
- **Affidabilità percepita dello scanner OCR:**
 - Media: $\mu = 4.5$
 - Varianza: $\sigma^2 = 0.34$
 - Deviazione Standard: $\sigma = 0.58$

I dati confermano che l'affidabilità dello scanner OCR non solo ha registrato la media più alta, ma anche la deviazione standard più bassa in assoluto ($\sigma = 0.58$), indicando un

consenso estremamente compatto. Per evidenziare graficamente le differenze di percezione tra le varie funzionalità, le distribuzioni delle frequenze assolute per tutte e tre le metriche analizzate sono illustrate negli istogrammi seguenti (Figure 21, 22 e 23).

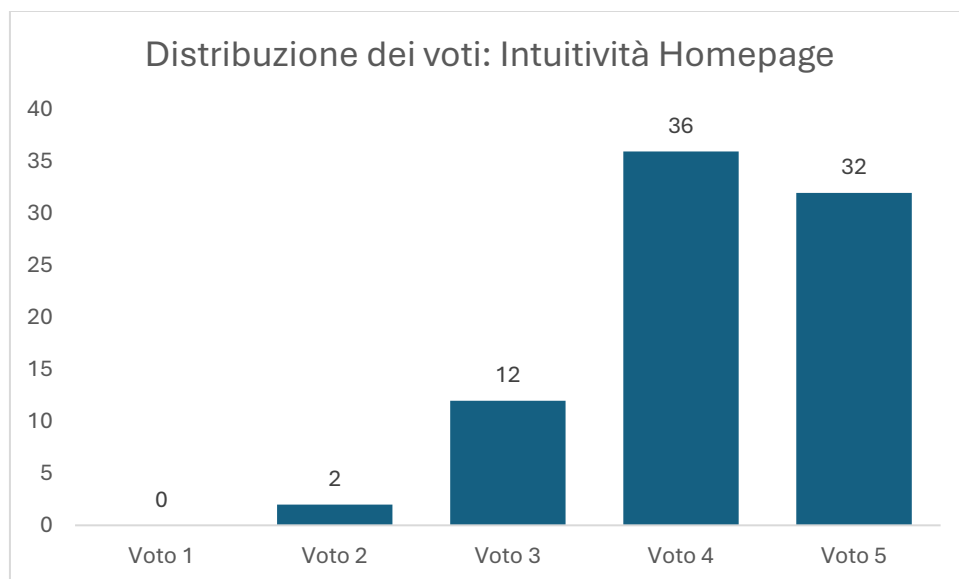


Figura 21: Istogramma della distribuzione dei voti sull'intuitività dell'interfaccia.

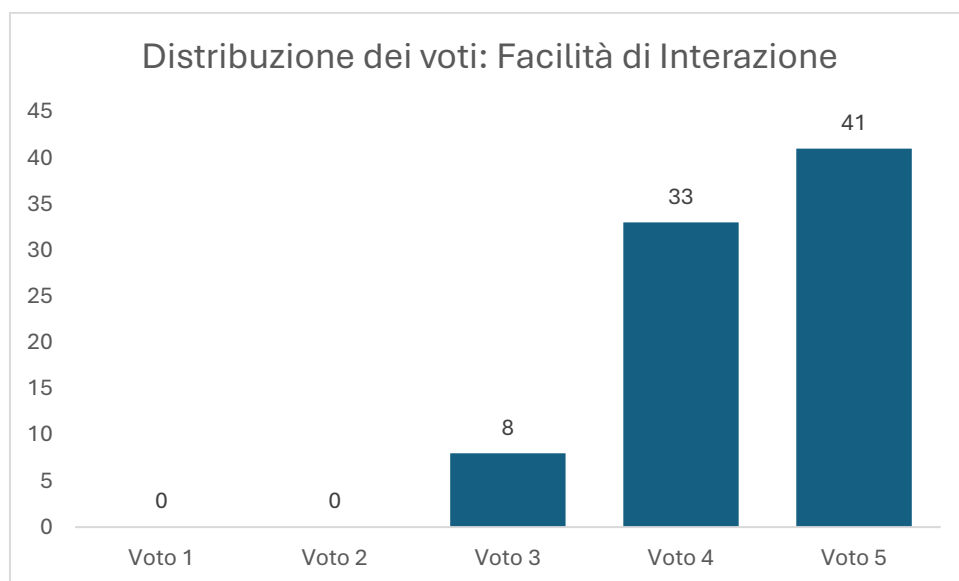


Figura 22: Istogramma della distribuzione dei voti sull'interazione e commento

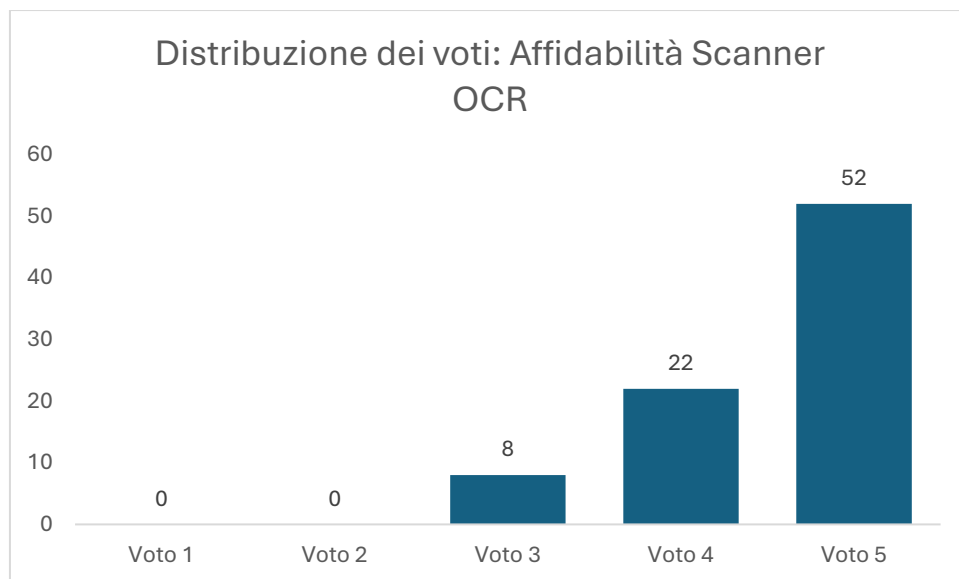


Figura 23: Istogramma della distribuzione dei voti sull'affidabilità dello Scanner OCR

Feedback Qualitativi e Sviluppi Futuri

L'analisi delle domande a risposta aperta ha fornito preziosi riscontri pratici. Di seguito alcuni esempi testuali dei feedback ricevuti:

- "La funzione di scansione è velocissima, mi permette di non dover ricopiare interi paragrafi dalle dispense cartacee."
- "Sarebbe utilissimo poter esportare i testi e i relativi commenti in un unico file PDF per stamparli."
- "A volte la fotocamera impiega qualche istante di troppo per la messa a fuoco in condizioni di luce artificiale scarsa."

Oltre all'esportazione in PDF, la roadmap per gli sviluppi futuri suggerita dall'utenza include la creazione di un client per ambiente Android e il potenziamento dei modelli di Machine Learning per supportare il riconoscimento testuale di lingue con alfabeti non latini (es. greco antico).

Conclusioni

Il presente lavoro di tesi ha documentato l'intero ciclo di vita di Stratum, un'applicazione iOS progettata per supportare e innovare le dinamiche dell'apprendimento collaborativo. Partendo dall'analisi delle criticità emerse durante la pandemia, il progetto si è evoluto per rispondere alle attuali esigenze del Blended Learning, offrendo uno strumento capace di unire lo studio sui supporti fisici tradizionali con le potenzialità della condivisione digitale.

Dal punto di vista ingegneristico, lo sviluppo ha confermato la validità e la robustezza delle tecnologie dell'ecosistema Apple. L'adozione del pattern architetturale Model-View-Controller (MVC), unita all'impiego del pattern Data Access Object (DAO), ha permesso di creare un codice sorgente modulare, manutenibile e fortemente disaccoppiato. L'integrazione di VisionKit ha dimostrato come la Computer Vision (OCR) possa abbattere i tempi di digitalizzazione dei documenti cartacei, mentre l'utilizzo nativo delle API di CloudKit ha garantito una sincronizzazione dei dati in tempo reale e sicura, priva di infrastrutture server di terze parti.

La fase conclusiva di validazione sul campo, condotta tramite la piattaforma TestFlight su un campione di oltre cento utenti, ha certificato l'affidabilità del sistema, registrando una percentuale di sessioni prive di crash (Crash-Free Sessions) del 98.2%. Contemporaneamente, l'analisi statistica del questionario di usabilità ha evidenziato un eccellente grado di soddisfazione da parte del target studentesco, confermando in particolar modo l'intuitività dell'interfaccia e la precisione dello scanner testuale integrato.

In prospettiva futura, il software possiede ampi margini di scalabilità. Come suggerito dagli stessi utenti durante la fase di beta testing, la roadmap di sviluppo prevede l'implementazione di un client multiplatforma (Android), l'integrazione di funzionalità per l'esportazione dei documenti elaborati in formato PDF e l'addestramento di modelli di Machine Learning personalizzati per estendere il riconoscimento OCR anche alle lingue con alfabeti antichi.

In conclusione, il progetto ha dimostrato come l'applicazione rigorosa dei principi dell'Ingegneria del Software, unita a un design incentrato sull'utente (UX), possa generare soluzioni concrete ed efficaci per supportare le nuove metodologie didattiche nel panorama EdTech contemporaneo.

Bibliografia

[1] Apple Inc. Swift Programming Language Documentation. [Online]. Disponibile all'indirizzo: <https://swift.org/documentation/>

[2] Apple Inc. CloudKit Framework Documentation. Apple Developer. [Online]. Disponibile all'indirizzo: <https://developer.apple.com/documentation/cloudkit>

[3] Apple Inc. Vision and VisionKit Frameworks: Recognizing Text in Images. Apple Developer. [Online]. Disponibile all'indirizzo: <https://developer.apple.com/documentation/vision>

[4] Apple Inc. Model-View-Controller in iOS. Cocoa Core Competencies. [Online]. Disponibile all'indirizzo: <https://developer.apple.com/library/archive/documentation/General/Conceptual/DevPedia-CocoaCore/MVC.html>

[5] Nichols, M., Cator, K., e Torres, M. Challenge Based Learning Guide. Digital Promise, 2016. [Online]. Disponibile all'indirizzo: <https://www.challengebasedlearning.org/>

[6] Sommerville, I. Ingegneria del software. Pearson, ottava edizione, 2007.

[7] SMOWL. (2025). *Educational technology trends: current and new trends*. [Online]. Disponibile all'indirizzo: <https://smowl.net/en/blog/educational-technology-trends/>

- [8] Kaopiz. (2025). *Top 11 Trends in Educational Technology to Watch in 2025*. [Online]. Disponibile all'indirizzo: <https://kaopiz.com/en/articles/trends-in-educational-technology/>
- [9] Softices. (2025). *How Education Industry Evolved with Technology Post-Pandemic*. [Online]. Disponibile all'indirizzo: <https://softices.com/blogs/edtech-industry-post-pandemic-evolution>
- [10] Universitat Oberta de Catalunya (UOC). *Digital transformation and pandemic: seven educational trends for the post-COVID-19 era*. [Online]. Disponibile all'indirizzo: <https://blogs.uoc.edu/elearning-innovation-center/digital-transformation-and-pandemic-seven-educational-trends-for-the-post-covid-19-era/>
- [11] Gaebel, M., & Zhang, T. (2024). *Innovazione didattica e ambienti inclusivi all'università*. Edizioni ETS. [Online]. Disponibile all'indirizzo: https://www.edizioniets.com/priv_file_libro/5323.pdf

Ringraziamenti

Sento il dovere di dedicare questo spazio del mio elaborato alle persone che hanno contribuito, con il loro instancabile supporto, alla realizzazione dello stesso.

In primis, un ringraziamento speciale al mio relatore, Prof. Francesco Isgrò, per la sua disponibilità durante la stesura di questa tesi.

Ringrazio i miei genitori che mi hanno sempre sostenuto, appoggiando ogni mia decisione, fin dalla scelta del mio percorso di studi.

Ringrazio tutte le persone che ho conosciuto durante gli anni accademici, ognuno di loro ha lasciato un segno nel mio percorso di crescita. In particolar modo il mio gruppo di studi formato da Felice, Gianni, Peppe, Raffaele, Roberto e Sasi sempre disponibili per qualsiasi problema e sempre pronti a soffrire e gioire insieme per ogni avvenimento.

Ringrazio inoltre tutte le persone che ho conosciuto durante la Apple Developer Academy e che mi hanno aiutato ad esplorare nuovi concetti a crescere personalmente. Un'esperienza che ha cambiato profondamente i miei punti di vista.

Ringrazio tutti i miei amici, in particolare Antonio, Luigi, Michele e Salvatore, che mi hanno sempre sostenuto e creduto in me.

Infine, ringrazio me stesso per aver continuato a crederci.